

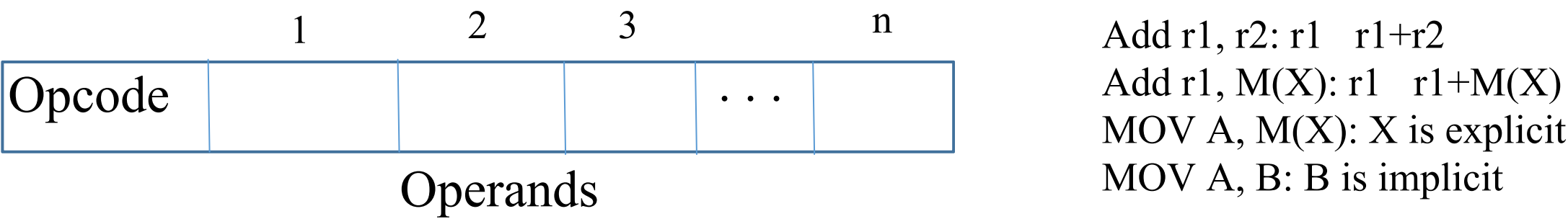
Instruction sets

Specify an operation to be carried out or set of operands or data to be used

Operation specified in opcode field of the instruction

Operand field includes input data and results that are produced

Operand field of the instruction contains addresses of storage locations in main memory or in the processor



Instruction format

n-operand format

To reduce instruction size and thereby reduce program storage space it is common to specify only $m < n$ operands explicitly in the instruction, remaining operands are implicit

Explicit address fields usually refer to main memory implicit ones refer to registers

If m is the maximum number of explicit main memory addresses allowed in any processor instruction, the processor may be called an m -address machine

Implicit input operands must be placed in locations known to the processor before the instruction that refers to them is executed

Addressing modes

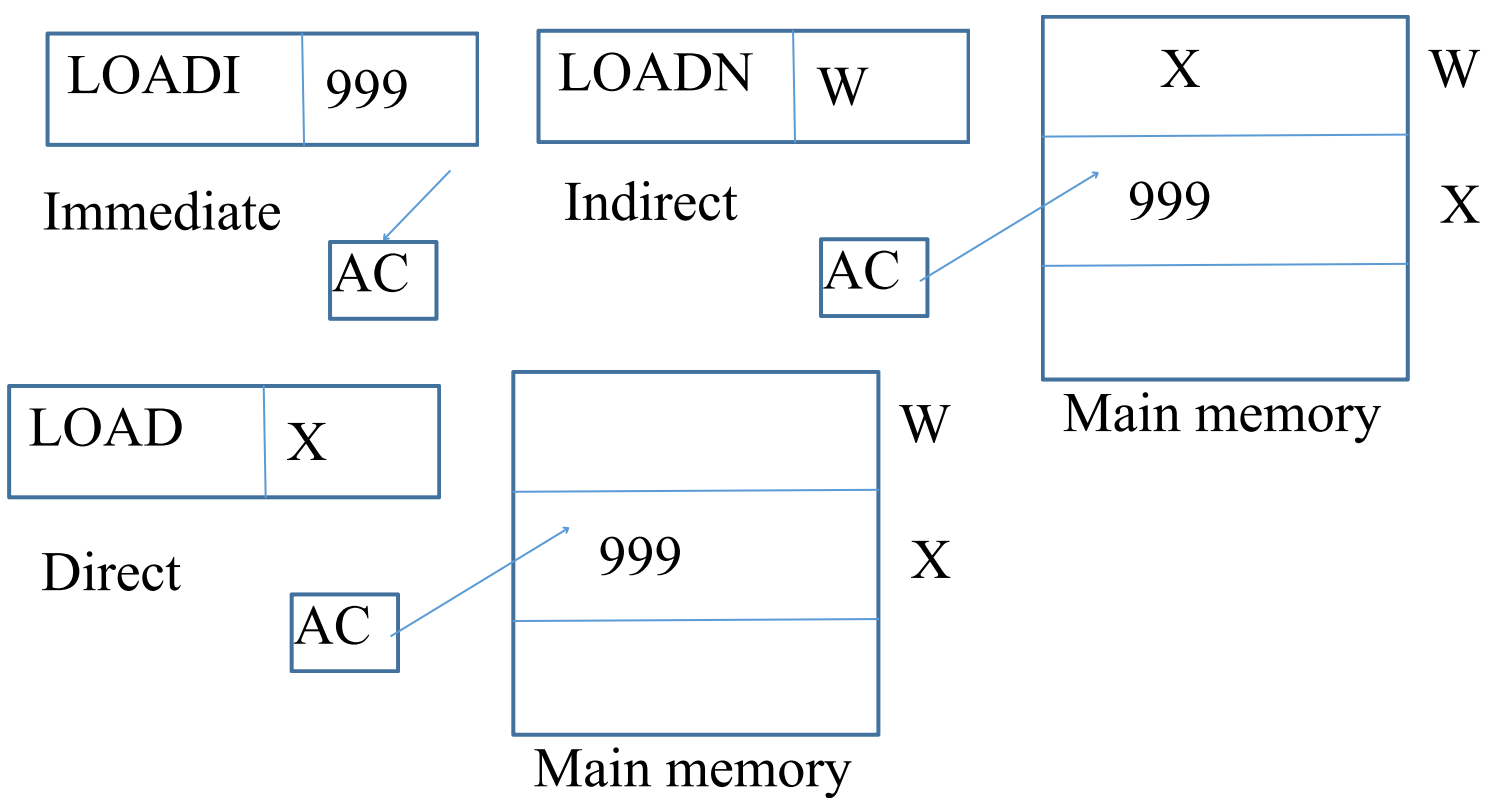
Operand field of an instruction contains a piece of data X

To execute instruction processor requires the current value of X , this value can be specified in several ways, termed addressing modes

Immediate addressing: X (Operand) is constant, its value can be placed in the instruction operand field

Direct addressing: X (Operand) is variable, then operand field X contains address of the memory location of the operand.
Requires one fetch operation for getting the operand.

Indirect addressing: Instruction operand field contains the address W of a storage location which in turn contains address X of the desired operand. By changing the content of W, address of the operand value required by the instruction is effectively changed
Requires two fetch operation for getting the operand.



Absolute addressing: Direct addressing needs the complete operand address

Partial addressing: Partial addressing information is included in the instruction, so the complete (absolute) operand address must be constructed by the processor

Relative addressing: Operand field contains a relative address or displacement D

Absolute or effective address (A) = Single address register from a set of general purpose registers associated with the operand (R) + Displacement (D)

Why relative addressing:

Instruction length is reduced as all the address information need not be included in the instructions

By changing the content of R, processor can change the absolute addresses referred to by a block (B) of instructions

Permits the processor to move (relocate) the entire block B from one region of memory to another without invalidating the addresses in B

When used in this way, R may be referred to as a base register and its content as a base address

R can be used for storing indices to facilitate the processing of indexed data, in this role, R is called index register
Indexed items $X(0), X(1), \dots, X(i)$ are stored in consecutive addresses in main memory. Instruction address field contains D the address of the first item $X(0)$ while the index register R contains the index i. Address of item $X(i)$ is $D+R$.
By changing the content of the index register, a single instruction can be made to refer to any item $X(i)$ in the given data list.
Disadvantage: Needs extra logic circuits and extra processing time for address computation

Autoindexing: Indexed items are frequently accessed sequentially so that a reference to $X(k)$ stored in memory location A is immediately followed by a reference to $X(k+1)$ or $X(k-1)$ stored in location $A+1$ and $A-1$ respectively.

To facilitate stepping through a sequence of items in this manner, addressing modes that automatically increment or decrement an address can be defined.

Example: $-(A3)$ Predecrement: indicates the content of A3 should be decremented automatically before the instruction is executed

$(A3)+$ Postincrement: indicates the content of A3 should be incremented automatically after the execution of the instruction

Stack addressing: Part of main memory M used as a pushdown stack

Logical sequence of items which are accessible from only one end referred to as the top of the stack

PUSH operation: Write, stores item at the top of the stack

POP operation: Read, removes item stored at the top of the stack

Changes the position of the stack top by the length of the operand pushed or popped

Stack pointer (SP): Address register

Stores the address of the last operand placed in the stack

Contains new stack top after PUSH/POP operation

Number of addresses

Three address machine: ADD Z,X,Y; $Z \leftarrow X+Y$; add the contents of the memory location X and Y, place the result in location Z

Two address machine: ADD X,Y; $X \leftarrow X+Y$; accumulator may be used to store the result, in that case $AC \leftarrow X+Y$;

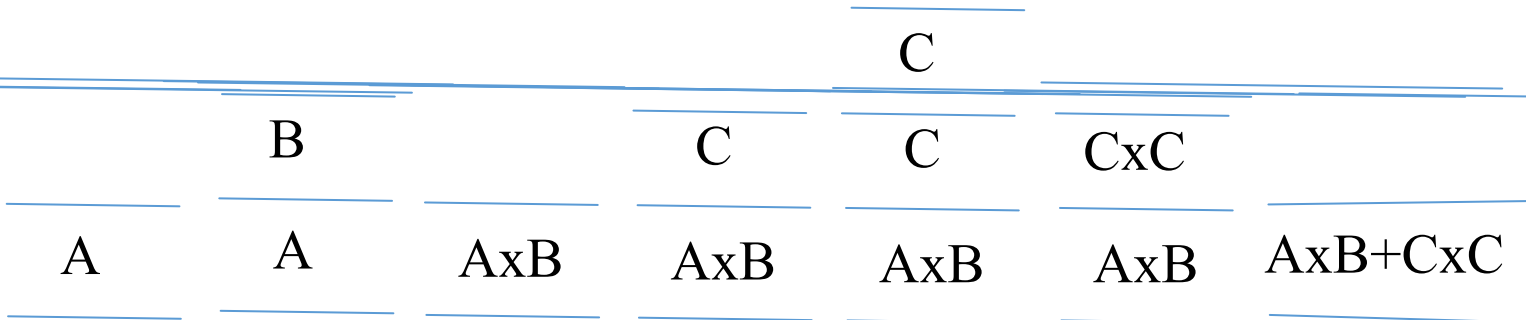
One address machine: ADD X; unspecified operands are assumed to be stored in fixed locations, commonly in a processor register AC; $AC \leftarrow AC+X$

Zero address machine: ADD; causes the two top operands X and Y to be removed from the stack and added; store the result at the top of the stack

One address machine	
LOAD A	$AC \leftarrow A$
MULTIPLY B	$AC \leftarrow AC \times B$
STORE T	$M(T) \leftarrow AC$
LOAD C	$AC \leftarrow C$
MULTIPLY C	$AC \leftarrow AC \times C$
ADD T	$AC \leftarrow AC + T$
STORE X	$M(X) \leftarrow AC$

Two address machine	
MOVE T,A	$T \leftarrow A$
MULTIPLY T,B	$T \leftarrow T \times B$
MOVE X,C	$X \leftarrow C$
MULTIPLY X,C	$X \leftarrow X \times C$
ADD X,T	$X \leftarrow X+T$

Three address machine	
MULTIPLY T,A,B	$T \leftarrow A \times B$
MULTIPLY X,C,C	$X \leftarrow C \times C$
ADD X,X,T	$X \leftarrow X+T$



Zero address machine	
PUSH A	Transfer A to top of stack
PUSH B	Transfer B to top of stack
MULTIPLY	Remove A, B from top of the stack and replace by $A \times B$
PUSH C	Transfer C to top of stack
PUSH C	Transfer second copy of C to top of stack
MULTIPLY	Remove C, C from top of the stack and replace by $C \times C$
ADD	Remove $C \times C$ and $A \times B$ from stack and replace by their sum
POP X	Transfer result from top of stack to X

Opcodes

In most computers, the opcode field is a fixed length field of k-bit within each instruction

Permits up to 2^k distinct operations

Some addressing information may also be included in the opcode field, for example, the addresses of processor registers used to store operands

Programs occupy a considerable amount of valuable storage space, so objective is to reduce their length as much as possible

Frequency with which operand addresses occur can not be reasonably determined; all may be assumed to have equal probability

Frequency with which specific instruction types occur can be determined but it depends on the instruction mixes

Attempt can be made to control the opcode length depending upon their probability of occurrence

Fixed opcode length: Easy to decode and store

Variable opcode length: Significant storage saving can result

Opcode having higher probability of occurrence in average application programs are encoded with fewer bits

Variable length opcode based on instruction occurrence probabilities		
Instruction	Probability of occurrence	Opcode Ci
I3	0.08	000

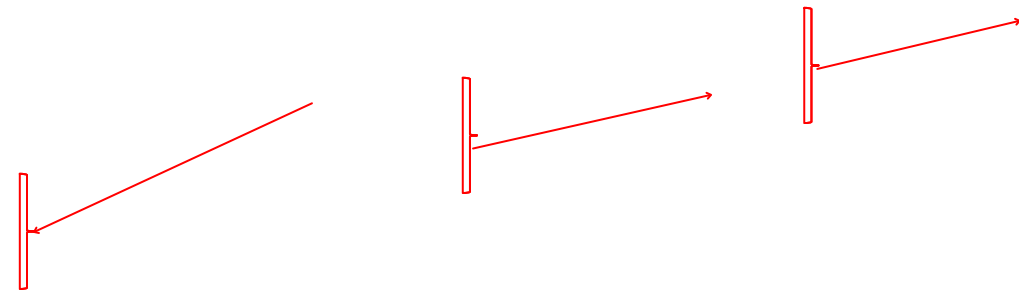
Fixed length opcode needs 3-bit per opcode

Variable length opcode needs average number of bits per instruction = $\sum_{i=1}^5 p_i |C_i|$

Choose each opcode carefully for unique decoding

% of saving = $\frac{3 - 1.82}{3} \times 100$

Huffman's method of optimal coding scheme								
	L1		L2		L3		L4	
pi,3	0.08	000	0.3	01	0.3	01	0.5	1
			0.12	001			0.5	0



A systematic method of constructing codes to minimize average opcode length

Opcode decoding: Scan any given instruction from left to right
An instruction beginning with 0100... must be of type I2

Instruction types: What type of instructions should be included in a general purpose processor's instruction set

Requirement to be satisfied by an instruction set

It should be complete in the sense that one should be able to construct a machine language program to evaluate any function that is computable using a reasonable amount of memory space

Instruction set should be efficient in that frequently required functions can be performed rapidly using relatively few instructions

It should be regular i.e. it should contain expected opcodes and addressing modes, e.g. if there is left shift, there should be a right shift

To reduce both hardware and software design cost, the instructions may be required to be compatible with those of existing machines, e.g., previous members of the same computer family

A "computer family" refers to a group of computers sharing similar architecture, design, and core components, allowing them to run the same software, like the IBM PC family

Instructions are conveniently divided into 5 major groups

1) Arithmetic instruction: Perform operations on numerical data

Example: Add, Subtract, Multiply, Divide, Absolute (replace operand by its absolute value), Negate (change sign of operand), increment, decrement

2) Logical instruction: Include Boolean or other nonnumerical operations

Example: AND, OR, NOT, EX-OR, Shift, Rotate, Convert (change data format from binary to decimal)

3) Data transfer instruction: Cause information to be copied from one location to another

Example: Move, Store, Load, Exchange (swap source and destination content), Clear (transfer word of 0's to destination), Set (transfer word of 1's to destination), Push, Pop

4) Program control instruction: Change the sequence in which programs are executed

Example: Jump, Jumpconditional, Jump to subroutine, Return, Execute (fetch operand from a specified location and execute instruction), Skip (increment PC to skip next instruction), Test (test specified condition based on outcome), Compare

5) I-O instruction: Cause information to be transferred between the processor or its main memory and external IO devices

Example: Input, Output, Start IO (transfer instruction to IOP to initiate IO operation), Test IO (transfer status information from IO system to specified destination), Halt IO (transfer instruction to IOP to terminate IO operation)

Note: A function returns a value to the calling code, while a subroutine performs a task without returning a specific value.

Reduced instruction set computer (RISC)

Developed at IBM in the mid 1970s

Relatively few instruction types and addressing modes

Fixed and easily decoded instruction format

Fast single cycle instruction execution

Hardware rather than micro program control

Memory access limited mainly to load and store instructions

Use of compilers to optimize object code performance

Most RISC instructions involve only register-to-register operations that are internal to the CPU

- Reduces execution time to single cycle

- Allows CPU cycle time and hardware complexity to be minimized

Complex instructions are implemented by routines due to the unavailability of the complex instructions in the instruction set

Complex instruction set computer (CISC)

A large number of instructions typically from 100 to 250 instructions

Some instructions that perform specialized task are used infrequently

A large variety of addressing modes from 5 to 20

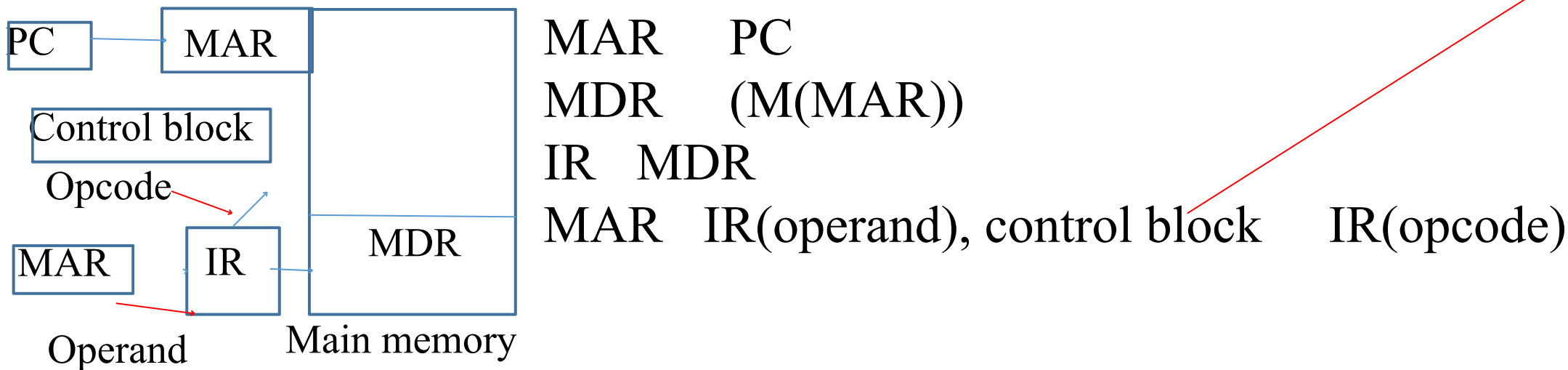
Variable length instruction format

Execution of instructions

Instructions are executed in instruction cycle

Instruction cycle has two parts, fetch and execute

Fetch cycle: PC contains the address of the next instruction to be executed



Control block starts to decode the instruction
CPU switches from fetch to execute cycle

Execution cycle

Execution of an instruction involves execution of a set of micro operations

Set of micro operations and their sequencing depends on the opcode and operand data type

Control block generates the sequence of control signals necessary to execute each of the micro operations during decoding

Example		
Instruction	Operation	Micro operations
LOAD X	AC ← M(X)	AR ← DR (operand part), Read M, DR ← MDR, AC ← DR

Arithmetic instruction implementation

Addition and subtraction with signed-magnitude data				
Operations	Add magnitudes	Subtract magnitude		
		When A>B	When A<B	When A=B
(+A)+(B)	+(A+B)			
(+A)+(-B)		+(A-B)	-(B-A)	+(A-B)
(-A)+(B)		-(A-B)	+(B-A)	+(A-B)
(-A)+(-B)	-(A+B)			
(+A)-(+B)		+(A-B)	-(B-A)	+(A-B)
(+A)-(-B)	+(A+B)			
(-A)-(+B)	-(A+B)			
(-A)-(-B)		-(A-B)	+(B-A)	+(A-B)

Add the two integer and attach the sign of A

Subtract B from A and attach the sign of A

Subtract A from B and attach the sign of B

Two equal numbers are subtracted, result +0 not -0

Hardware implementation

A and B be two registers to hold the magnitude of the numbers

As and Bs are two flip-flops to hold the sign of A and B

Result of the operation may be transferred to a third register or into A and As

A and As together form an accumulator register

Parallel adder performs A+B

Comparator to compare the magnitudes of A and B

Two parallel subtractor circuit to compute A-B and B-A

XOR gate to find sign relationship with As and Bs

Addition

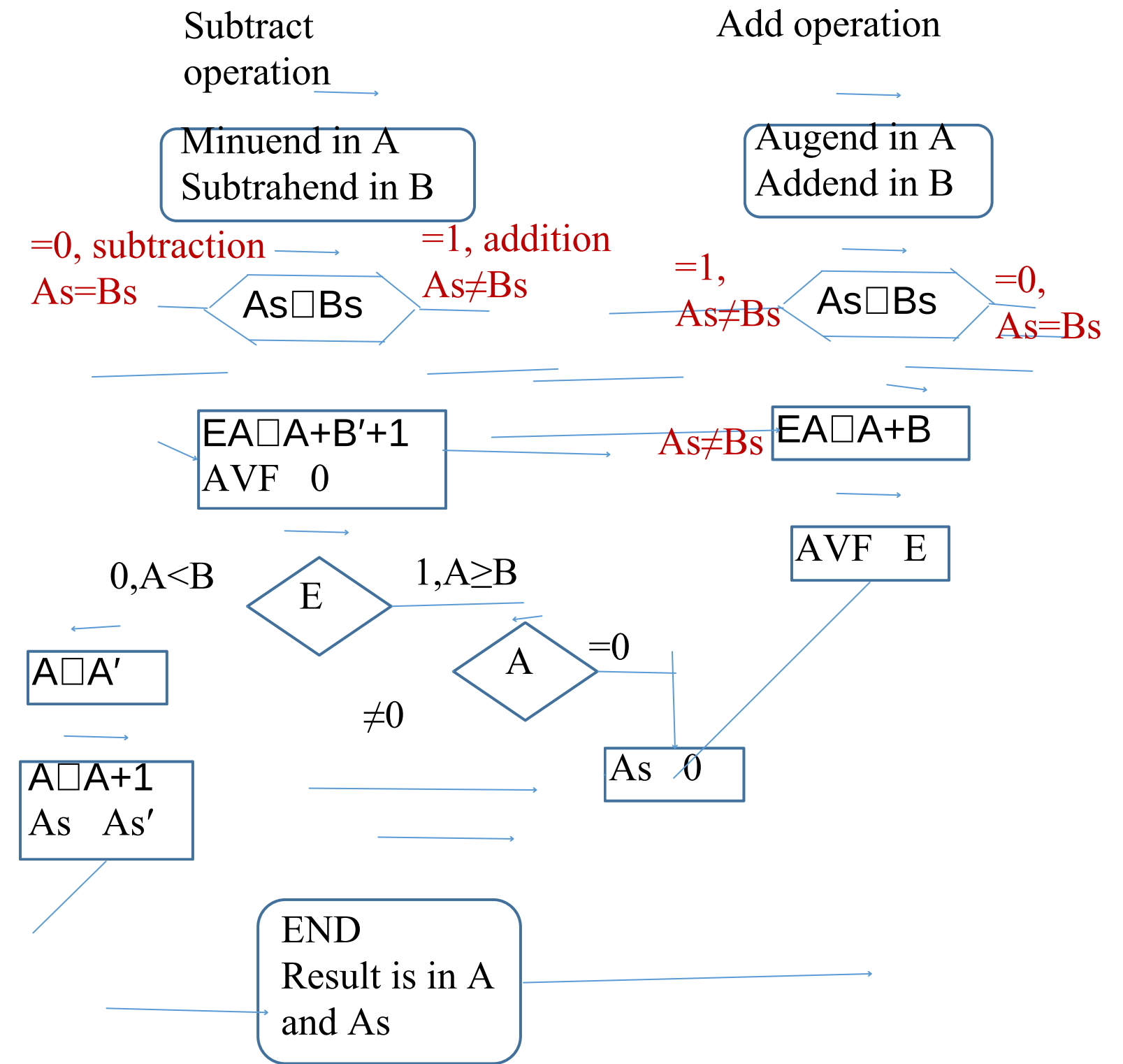
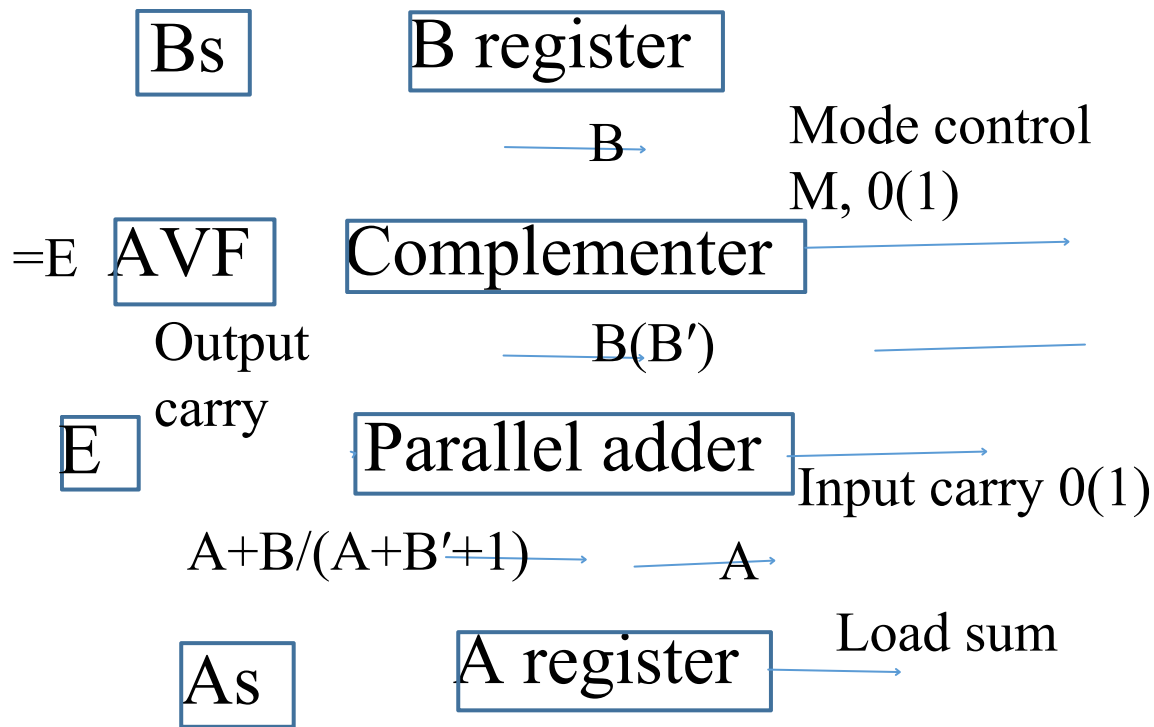
As=Bs, Addition and As ≠ Bs, Subtraction

Subtraction

As=Bs, Subtraction and As ≠ Bs, Addition

Procedure to minimize hardware equipment

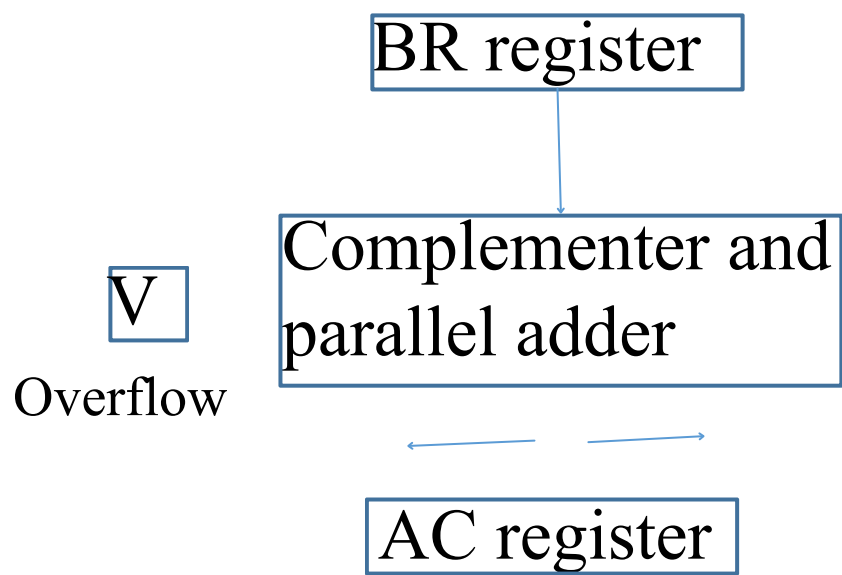
- Subtraction can be done using complement and addition
- Result of comparison can be determined from the end carry after the subtraction
- Use of 2's complement for subtraction and comparison is an efficient procedure as it requires only one adder and one complementer
- Subtraction can be done by adding A with 2's complement of B
- Output carry is transferred to E for comparison ($A > B$, $A < B$, $A = B$)
- AVF holds overflow bit when A and B are added
- Complementer consists of XOR gates
- Parallel adder consists of full adder
- Addition $\square M=0$, input carry = 0, adder output = $A+B$
- Subtraction $\square M=1$, 1's complement of B is applied to the adder, input carry = 1, output = $A+B'+1=A-B$



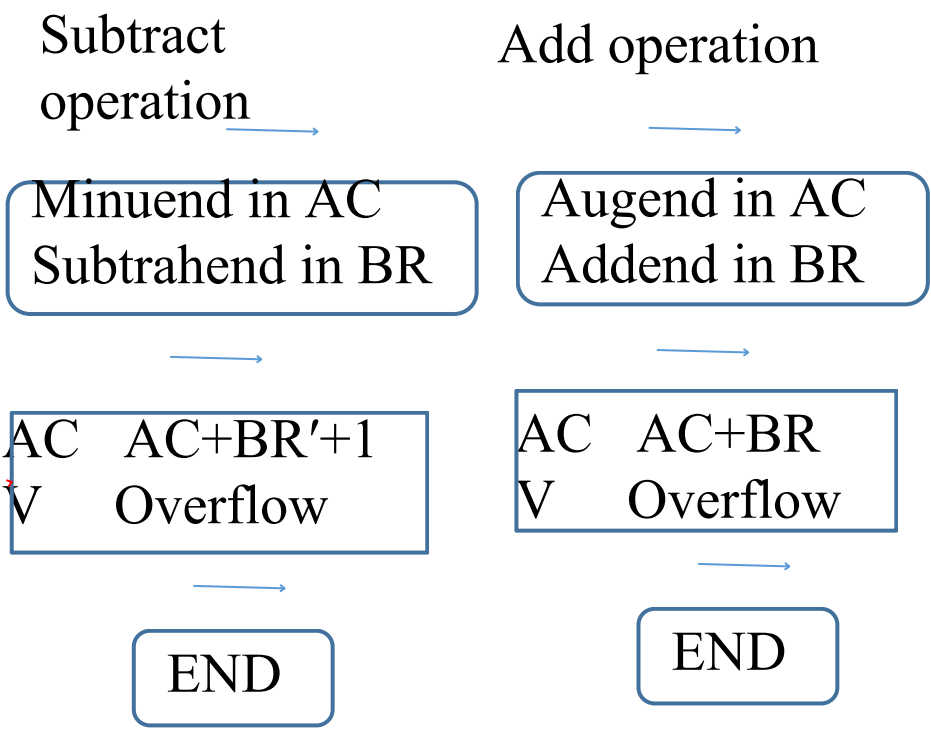
Addition and subtraction with signed 2's complement

Left most bit of a number represent the sign bit
Negative number is represented in 2's complement form
Overflow

Result of addition of two n-bit number has n+1 number of bits
Can be detected by the last two carry out of the addition



AC: A register
BR: B register
Left most bit in AC and BR represent the sign of the numbers
Two sign bits are added or subtracted together with the other bits in the complementer and parallel adder
Overflow flip-flop V is set to 1 if there is an overflow, output carry is discarded in this case
In the addition of two unsigned number an overflow is detected from the end carry. In case of signed number sign bit is treated as part of number and so end carry does not indicate overflow



Adds AC and 2's complement of BR
Overflow must be checked because two numbers added could have the same sign
In case of overflow there is an erroneous result in AC

Sum = AC+BR including their sign bits
V=1 if XOR of the last two carries is 1 otherwise it is cleared to 0
Two sign bits (As, Bs) are added/subtracted together with the other bits
2's complement of BR change its sign and so there is a chance of overflow during subtraction and AC contains wrong result

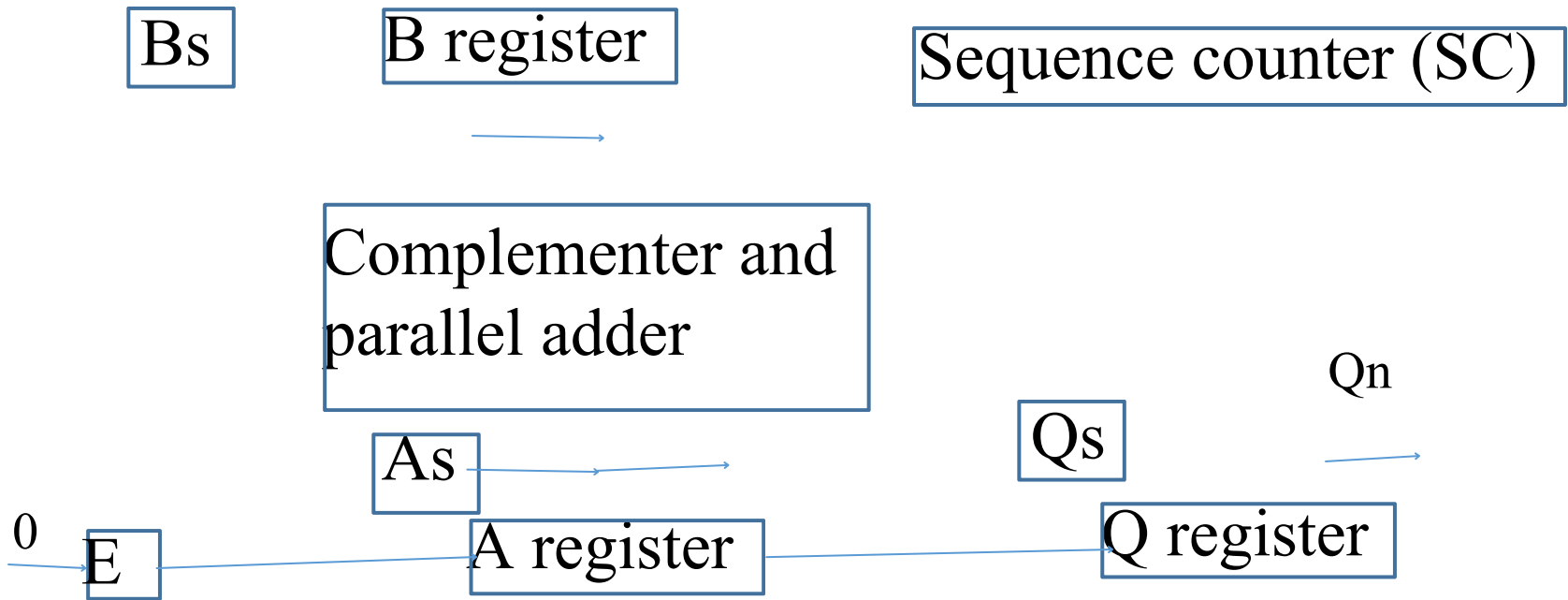
Multiplication algorithm

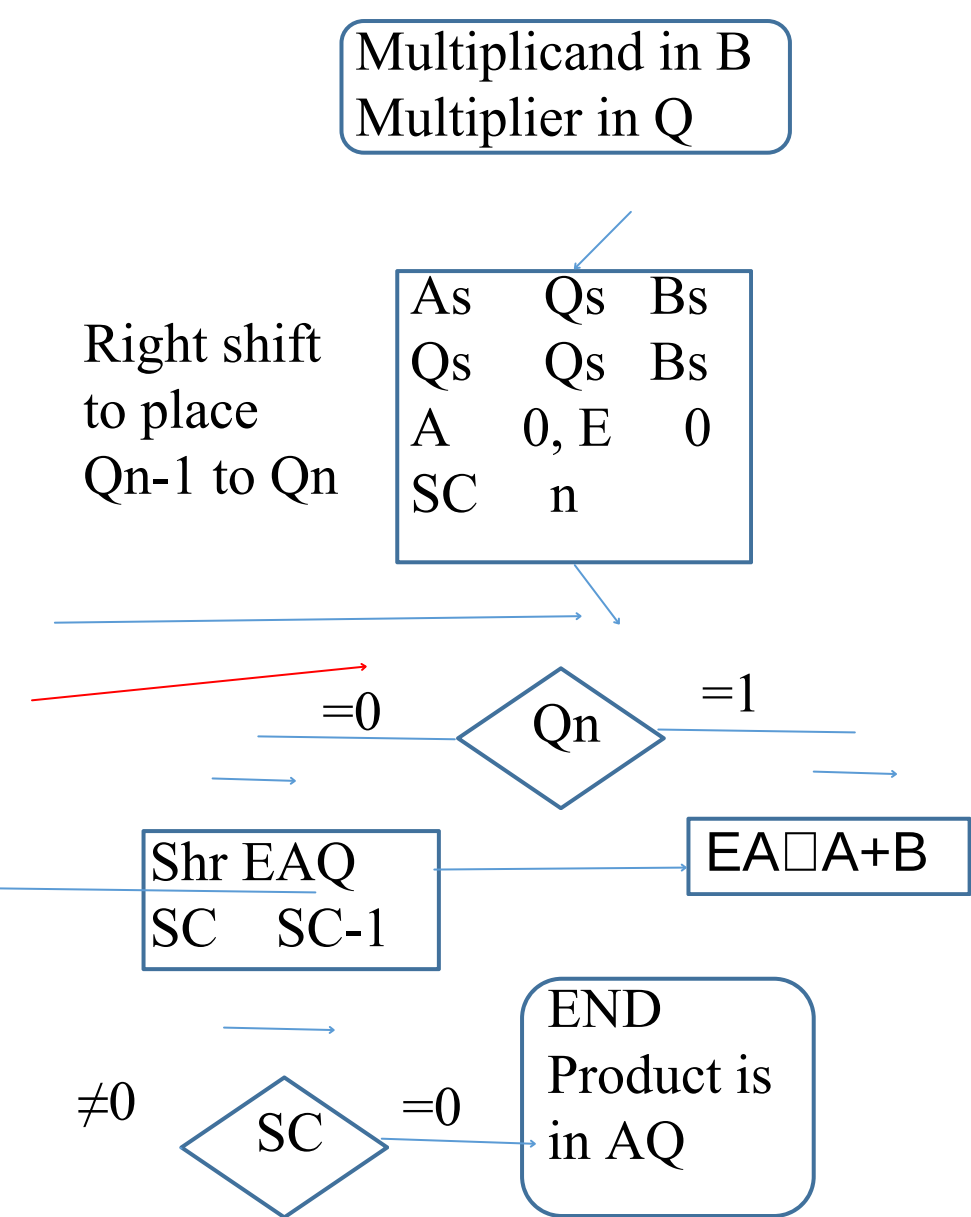
				1	0	1	1	1	Multiplicand
				1	0	0	1	1	Multiplier
				1	0	1	1	1	Partial product
			1	0	1	1	1		Partial product
		0	0	0	0	0			Partial product
	0	0	0	0	0				Partial product
1	0	1	1	1					Partial product
1	1	0	1	1	0	1	0	1	Sum of partial product

If multiplier bit is 1, copy multiplicand after shifting one bit left
Else copy 0 after 1 bit left shifting
Add the partial products

Hardware implementation for sign magnitude data

Adder for adding the partial products and store the result of multiplication in a register
Instead of shifting multiplicand towards left, partial product can be shifted towards right
Store multiplier in Q register and sign of multiplier in Qs
Store multiplicand in B register and sign of multiplier in Bs
SC = number of bits in multiplier initially, decrement by 1 after forming each partial product, process stops when SC is 0





Multiplicand B = 10111, Multiplier Q = 10011				
Operation	E	A	Q	SC
Initialization	0	00000	10011	101
Qn = 1, add B		10111		
1 st partial product	0	10111	10011	
Shr EAQ	0	01011	11001	100
Qn=1, add B	0	10111		
2 nd partial product	1	00010	11001	
Shr EAQ	0	10001	01100	011
Qn=0, shr EAQ	0	01000	10110	010
Qn=0, shr EAQ	0	00100	01011	001
Qn=1, add B		10111		
Shr EAQ	0	01101	10101	000

Booth's multiplication algorithm

Speed up the multiplication process

Multiply binary integers in signed 2's complement representation

String of 0's in the multiplier require no addition but just shifting

A string of 1's in the multiplier from bit weight 2^k to weight 2^m can be treated as $2^{k+1}-2^m$

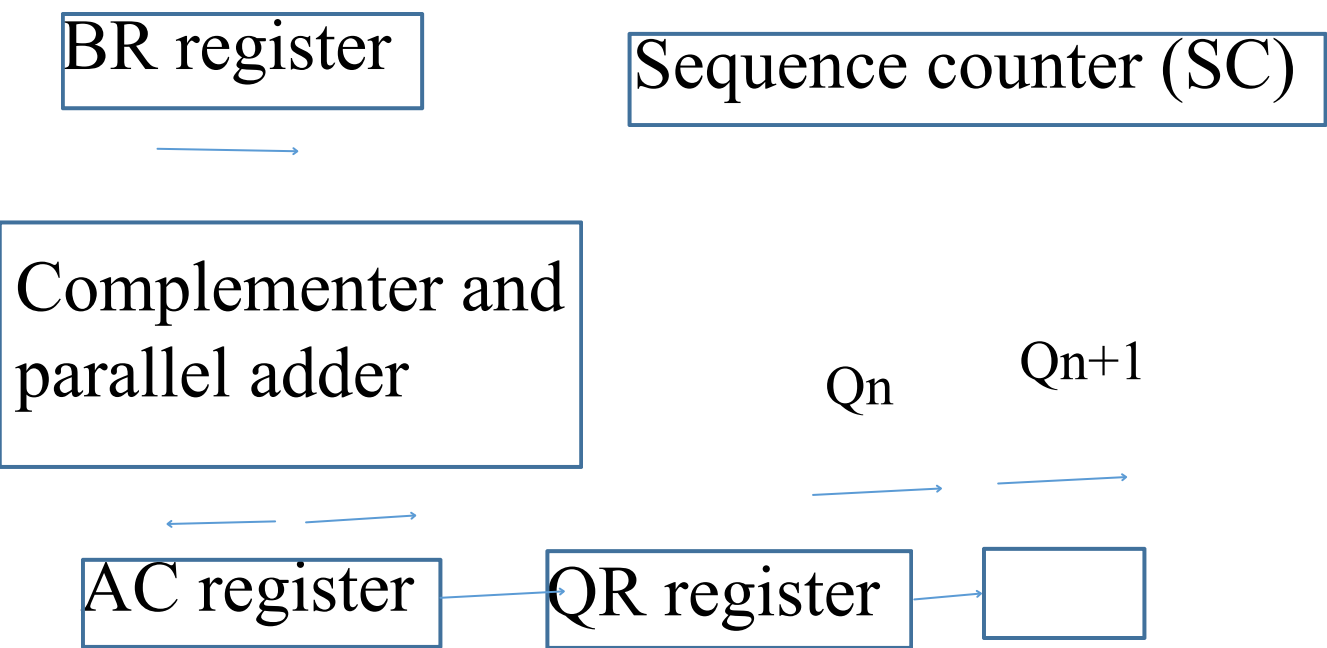
Example: $001110 = +14$ has a string of 1's from 2^3 to 2^1 , $k=3$ and $m=1$. So $+14$ can be represented as $2^{k+1}-2^m = 2^4-2^1 = 14$

Multiplication

M=multiplicand and $+14$ = Multiplier

$M \times 14$ can be done as $M \times 2^4 - M \times 2^1$

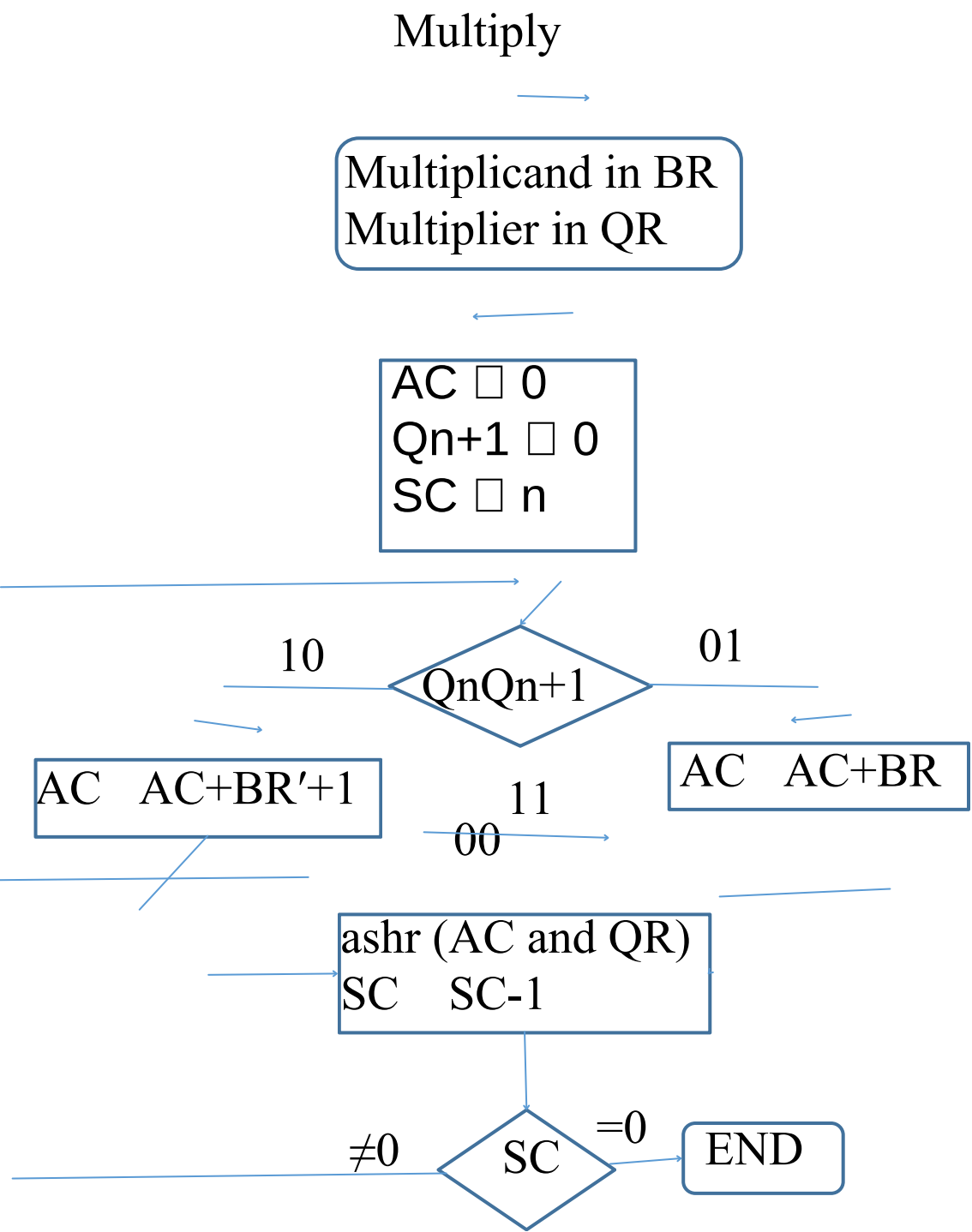
Product can be obtained by shifting the binary multiplicand M four times towards left and subtracting M after shifting left by 1 bit

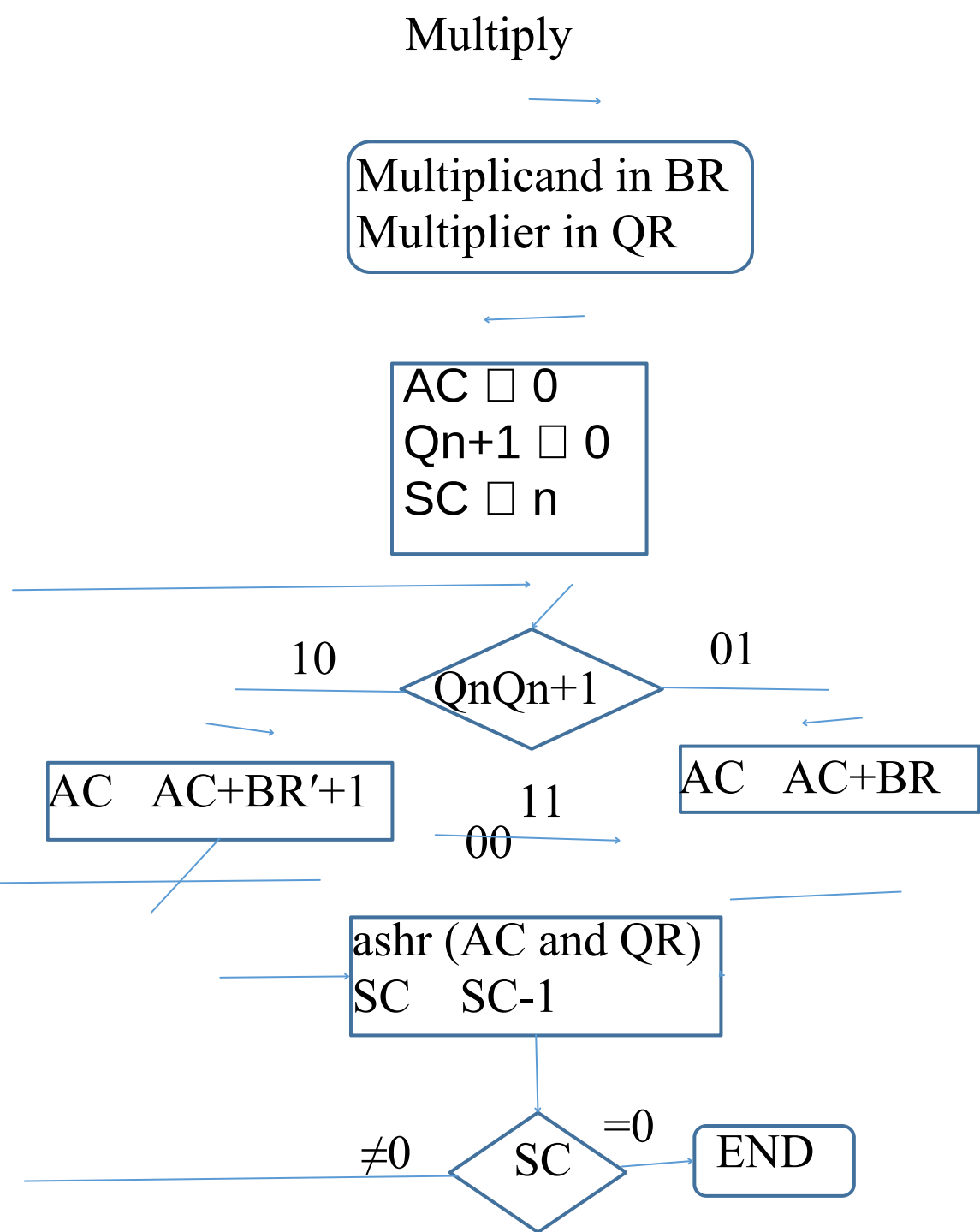


Hardware implementation

Q_n : LSB of multiplier in QR

Q_{n+1} : extra FF





Example: (-9) x (-13) = +117

Multiplicand in BR = -9 = 11001, 2's complement of 11001 is 10111

Multiplier in QR = -13 = 11101, 2's complement of 11101 is 10011

BR=10111					
QnQn+1	BR'+1=01001	AC	QR	Qn+1	SC
	Initialization	00000	10011	0	101
10	Subtract BR	01001			
	ashr	00100	11001	1	100
11	ashr	00010	01100	1	011
01	Add BR	10111			
	ashr	11100	10110	0	010
00	ashr	11110	01011	0	001
10	Subtract BR	01001			
	ashr	00011	10101	1	000

Division algorithm: Division of two fixed-point binary numbers in signed magnitude representation is done

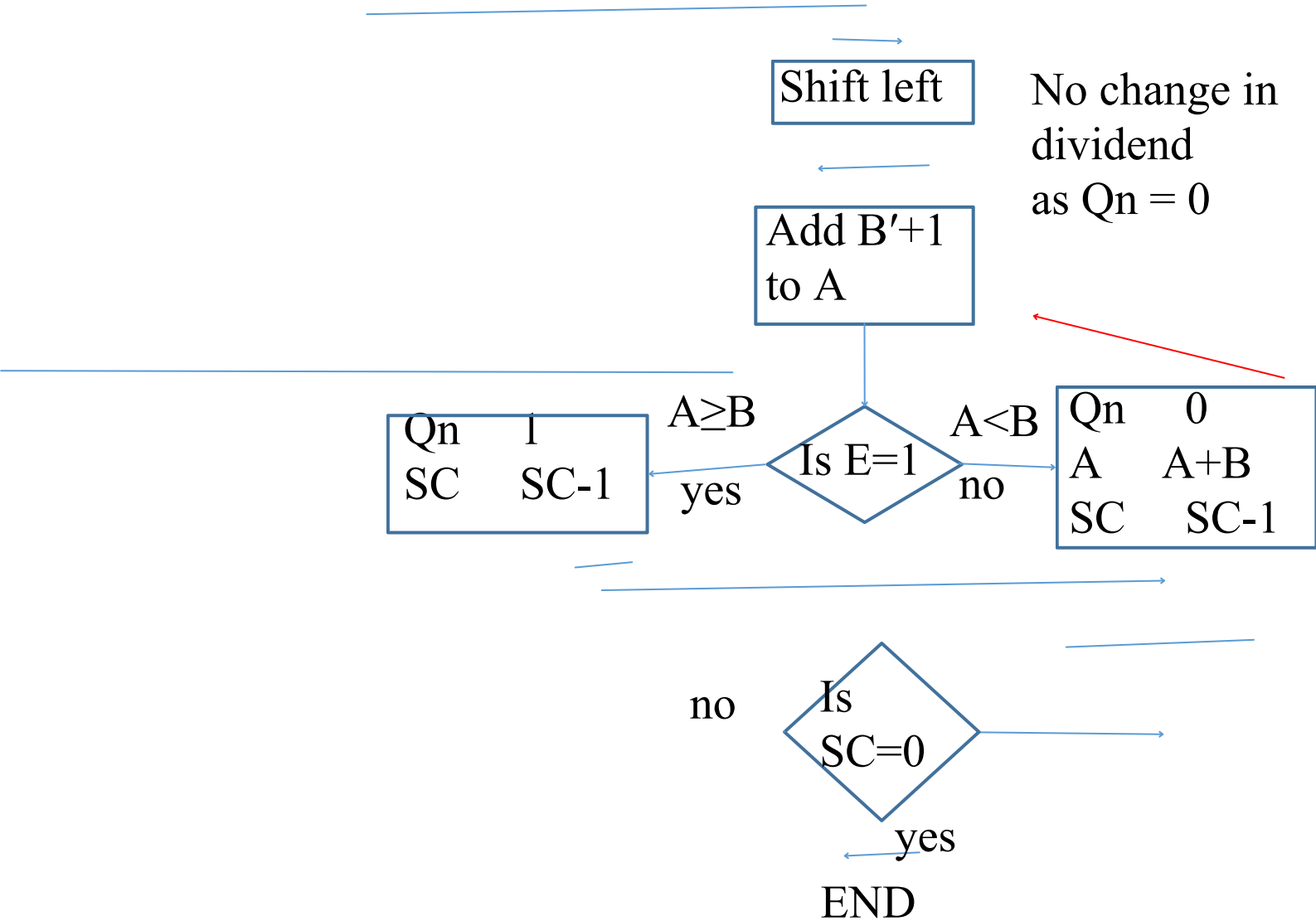
Divisor B consists of 5 bits, dividend A consists of 10 bits, five most significant bits (MSBs) of dividend are compared with the divisor

B = 10001, A = 01110 00000

Most significant 5 bits of A = 01110, less than divisor; so compare B and 6 MSBs of A

11010	Quotient Q
10001)0111000000 011100 10001	Dividend A 5 bits of A < B, so consider 6 bits of A Shift right B and subtract, enter 1 in Q
010110 10001	6 bits remainder > B Shift right B and subtract, enter 1 in Q
001010 00000	Remainder < B, enter 0 in Q, shift right B
010100 10001	Remainder >B Shift right B and subtract, enter 1 in Q
000110 00000	Remainder <B, enter 0 in Q
110	Final remainder

Sign of the result: Sign of quotient is determined from the sign of the dividend and the divisor. If the two signs are alike, sign of the quotient is plus. If they are unlike, sign is minus. The sign of the remainder is the same as the sign of the dividend.

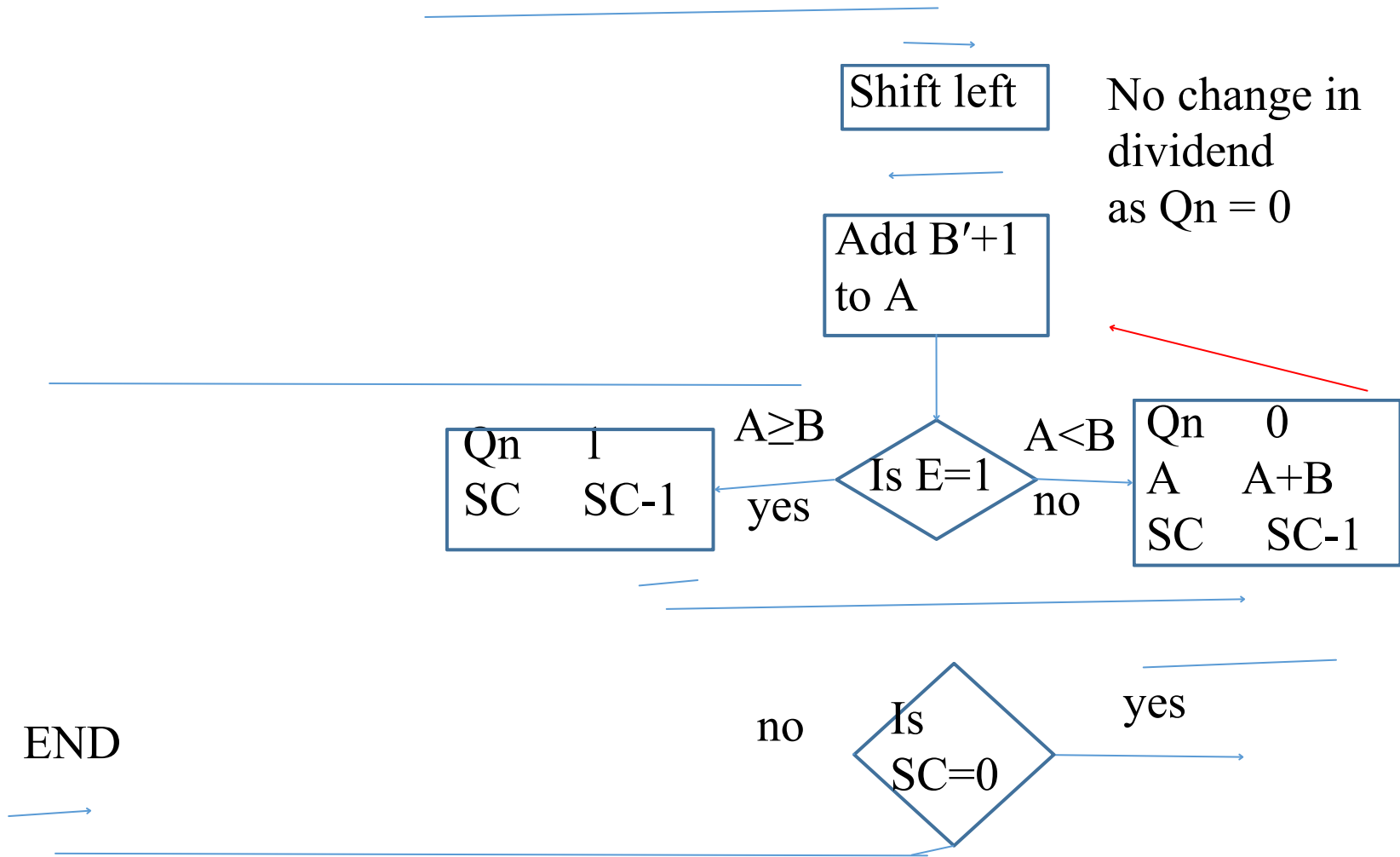
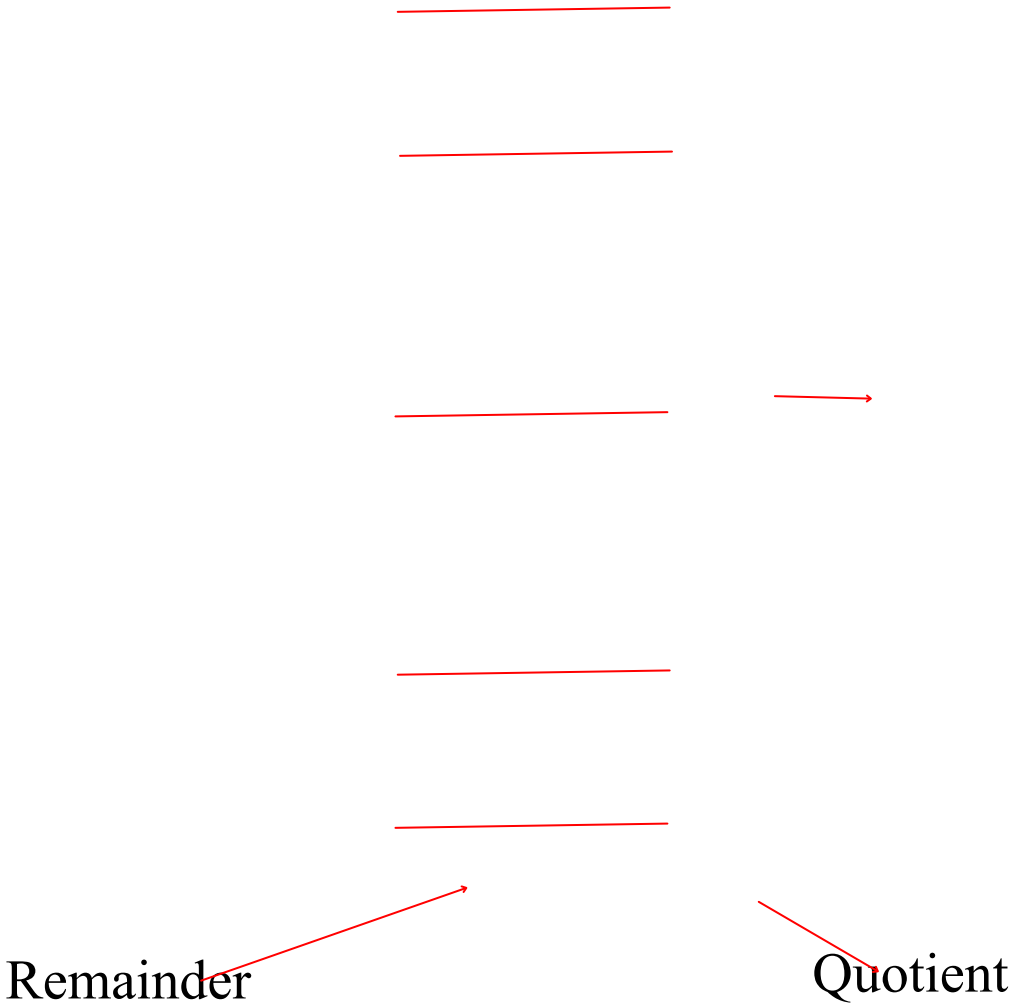


Example: Divisor B = 10001, B'+1 = 01111				
Operation	E	A	Q	SC
Shl EAQ	0	11100	00000	
Add B'+1		01111		
Add B'+1		01111		
E = 0, Qn = 0	0	11001	00110	
Add B'+1		01111		
E=0, Qn=0	0	10101	11010	

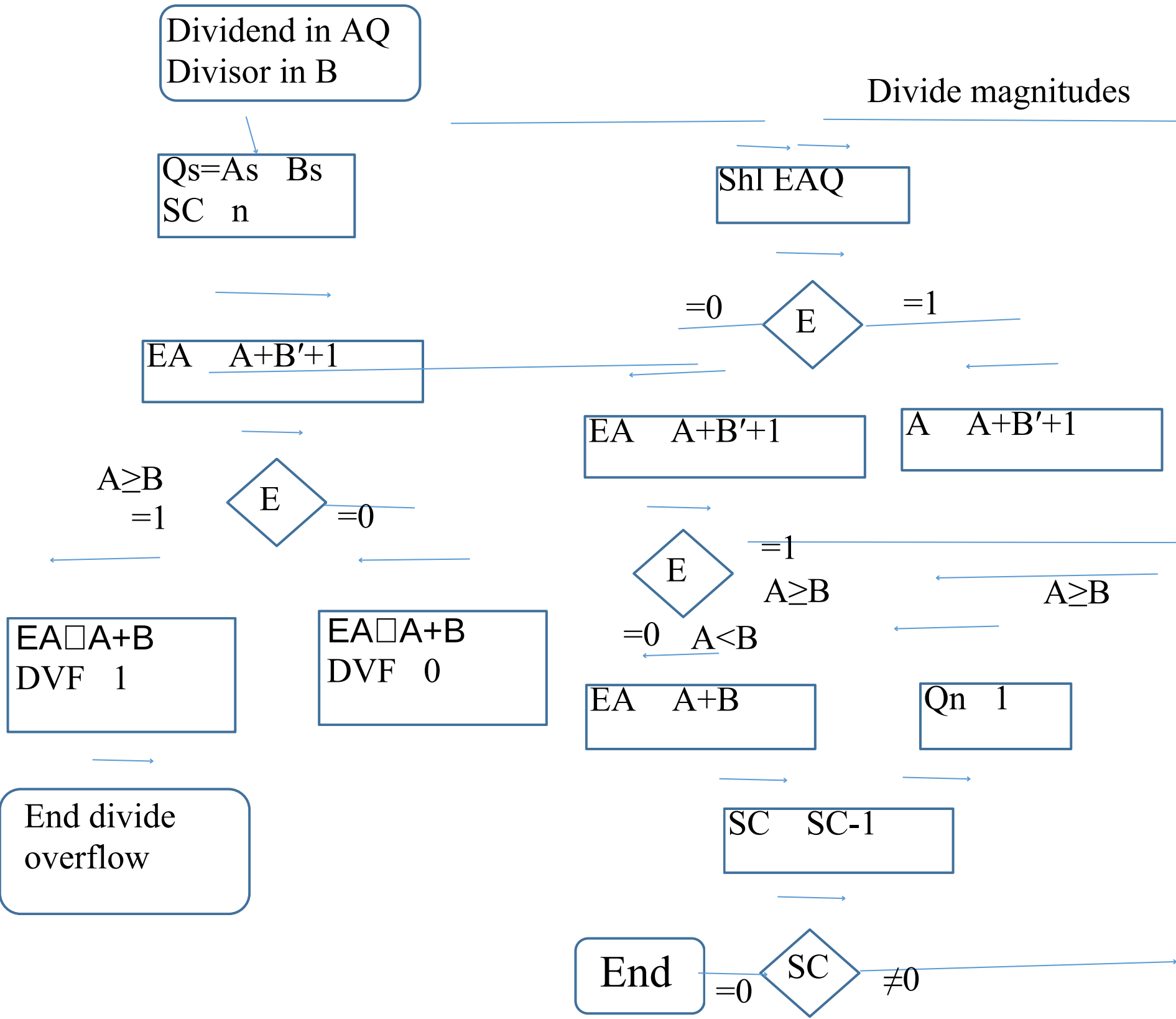
E = end carry information about the relative magnitude of A and B
 = 1 if $A \geq B$ and quotient, $Q_n = 1$
 = 0 if $A < B$ and quotient $Q_n = 0$, value of B is added to restore the partial remainder in A to its previous value

Left shifting: Partial remainder is shifted towards left, quotient bits are also shifted, after 5 shifts quotient is in Q, remainder is in A

Divide overflow: Division operation may result in a quotient with an overflow
 Consider a system with 5 bits registers. If 5 MSBs of $A > B$, size of the quotient becomes 6 bits, causes overflow, divide overflow flag (DVF) is set



Divide operation



Example: Divisor B = 10001, B'+1 = 01111				
Operation	E	A	Q	SC
Initialization		01110	00000	101
Shl EAQ	0	11100	00000	
E=0, add B'+1		01111		
E=1, Qn=1	1	01011	00001	100
Shl EAQ		01111		
E=0, add B'+1		01111		
E=0, add B'+1	0	11001		
Shl EAQ		01111		
E=0, add B'+1		01111		
E=0, add B'+1		10101		

Quotient

Dividend = 448
Divisor = 17
Quotient=26
Remainder = 6

Remainder